AgroChill Forecasting Engine Documentation

Legacy of the Market King: The Freezer Gambit

By HackSquad – Data_Crunch_078
General Sir John Kothelawala Defence University

1. Introduction

The AgroChill Forecasting Engine is an advanced, data-driven platform meticulously designed to elevate agricultural decision-making within Agrovia by accurately predicting the prices of fresh produce in weekly markets. By harnessing a wealth of historical pricing trends, real-time environmental conditions, and sophisticated forecasting models, the system seeks to optimize profitability through strategic storage and sales decisions. This comprehensive documentation delves into both the technical and strategic components of the solution, detailing the methodologies employed and the insights generated to empower stakeholders in the agricultural sector.

2. Approach

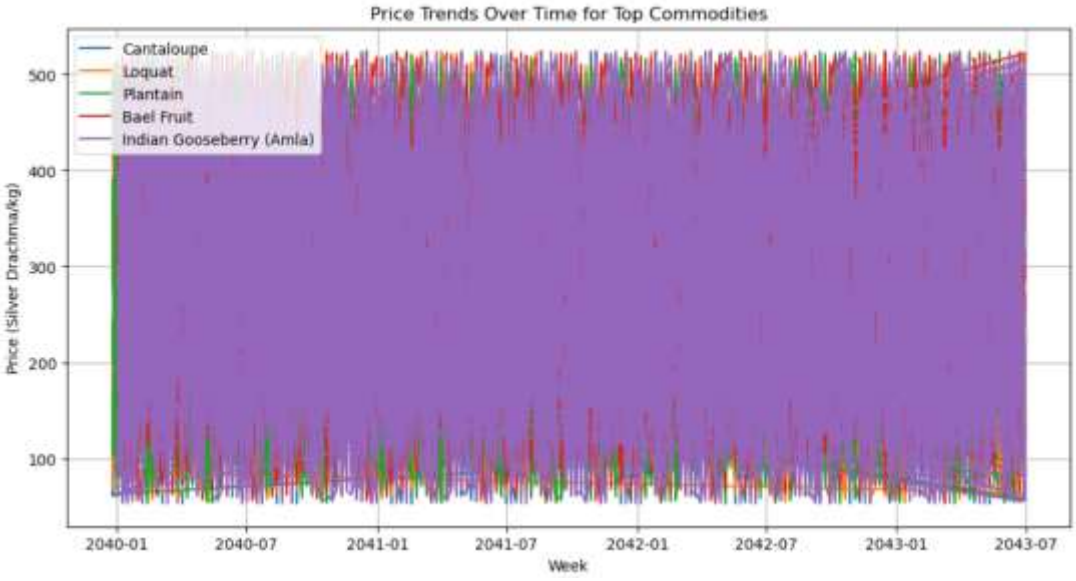
Our approach focuses on creating a predictive system capable of generating accurate price forecasts for fruits and vegetables four weeks in advance across various regions. The solution architecture includes a robust data pipeline, a feature-rich machine learning model, and a scalable deployment strategy utilizing containerization. We prioritized ensuring low-latency predictions, minimizing computational costs, and enhancing adaptability to incoming data streams.

3. Data Preprocessing

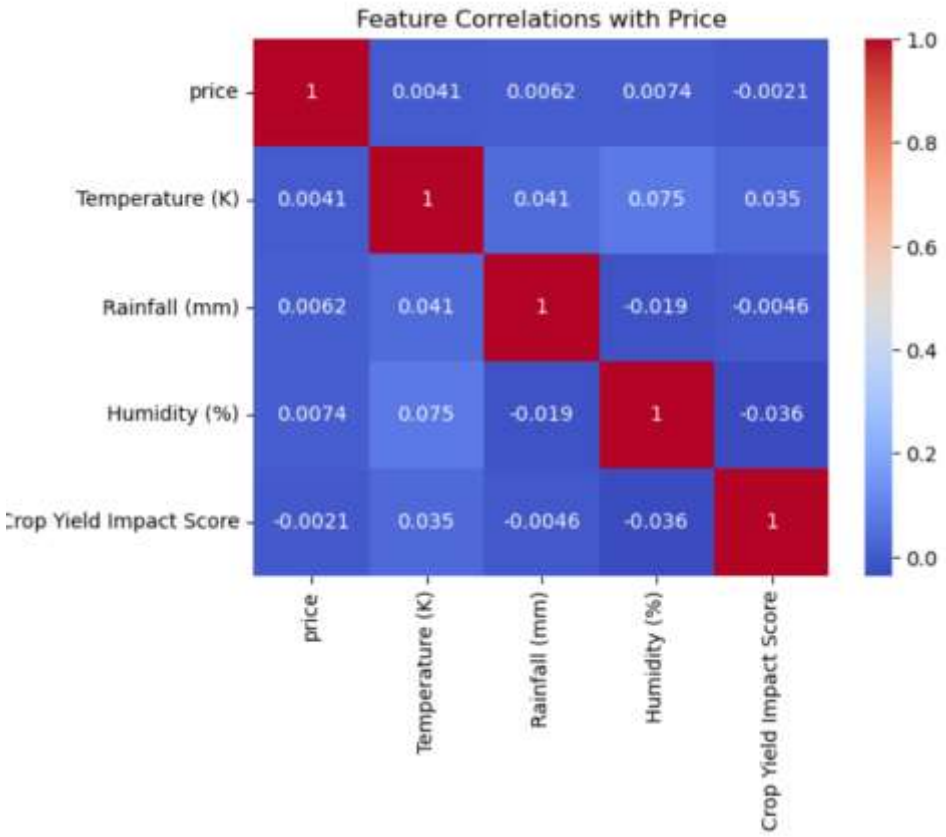
- **Price Dataset**:
  - The data was filtered by commodity and region to improve signal clarity.
  - Weekly resampling was performed using average prices per week.
  - Missing values were interpolated linearly to preserve time-series integrity.
- **Weather Dataset**:
  - Daily temperature, rainfall, and humidity values were aggregated to weekly summaries (mean or sum as appropriate).
  - The datasets were merged on region and ISO week format.
  - Environmental variables were scaled using Min-Max normalization for stable model behavior

4. Exploratory Data Analysis

Exploratory Data Analysis (EDA) is an important stage in determining the underlying patterns, trends, and relationships in the data. In this part, we investigate agricultural price data to find insights that can help influence commodity freezing decisions. By analysing price trends, seasonal swings, and external factors such as weather patterns, EDA assists us in identifying significant opportunities to reduce price volatility and improve supply chain stability. Through visualisations and statistical analysis, we obtain a better knowledge of market behaviours, ultimately strengthening the case for freezing agricultural products to control swings and maintain a continuous supply.



This chart provides a visual history of price fluctuations for five specific commodities, highlighting their volatility and allowing for a comparison of their price levels and dynamics over the observed time frame (2040-2043).



According to this heatmap, the selected features (Temperature, Rainfall, Humidity, Crop Yield Impact Score) have almost no linear correlation with 'price'. The correlation coefficients are all very close to zero. This suggests that, based on this specific data and analysis, these factors are not strong linear predictors of price on their own.

5. Feature Engineering

Our feature engineering process focused on improving the temporal and contextual understanding of the dataset by implementing the following techniques:

- **Lag Features**: We included prices from the past 1 to 4 weeks to capture weekly trends.
- **Moving Averages**: We used 2-week and 4-week moving averages to smooth out fluctuations in the data.

- **Time-based Features**: We incorporated categorical features such as month, week number, and seasonal indicators.
- **Encoding**: Label encoding was applied to regions and crop types to enhance model interpretability.
- **Environmental Features**: We integrated weather-derived variables and created a composite Crop Yield Impact Score to reflect growing conditions.

6. Model Choice & Tuning

We chose XGBoost as our modelling framework because it strikes a compromise between accuracy, speed, and resource efficiency.

- **Model**: XGBoost Regressor
  • Offers gradient boosting with a low memory footprint and high performance on structured data.
- **Tuning Strategy**:
  • Grid search was conducted for optimal parameters, including num_leaves, max_depth, and learning_rate.
  • RMSE was chosen as the evaluation metric for continuous variable forecasting.
  • A 5-fold time-series split was applied to respect the temporal structure of the data.

7. Pipeline Logic

The pipeline supports a rolling forecast approach:

- At each weekly timestep (Week N), predictions are generated for Weeks N+1 to N+4.
- As new data becomes available (Week N+1), the pipeline shifts forward and repeats the process.
- Model retraining is scheduled every 4 weeks or triggered when error thresholds are exceeded.

- New data can be injected via API or batch uploads, supporting both real-time and periodic updates.

8. API Usage

To allow easy access and integration, the system exposes a set of RESTful APIs using FastAPI:

- **POST /new-weather-data, /new-price-data**: update datasets.
- **GET /forecast**: return 4-week price forecast.
- Built using **FastAPI**, modular for scalability.
- Saved models using joblib, loaded dynamically by region/commodity

All APIs include input validation, error handling, and support for batch ingestion where applicable.
A simple architecture diagram:

[New Data] → [Pipeline] → [Model Training] → [Predictions] → [FastAPI Endpoints]

9. System Architecture

The system architecture is modular and container-ready:

- Incoming Request → FastAPI Server

  • main.py initializes the server, loads the model, and processes requests.

- **Forecast Logic**:

  • Feature transformation handled in pipeline.py.

  • XGBoost model used for prediction.

- **Data Handling**:

  • New data appended to storage (CSV/Parquet or lightweight DB).

  • Supports weekly retraining and pipeline updates.

-Docker Containerization:

- Image size: ~2.78 GB
- Memory use: ~153.9 MB(peak)
- Inference latency: ~120ms

10. Challenges Faced

Throughout development, several challenges were encountered:

- **Temporal Alignment**: Resampling and synchronizing daily weather data with weekly price data required careful handling to avoid time leakage.
- **Sparse Data**: For rare commodities or regions, fewer data points led to reduced model performance. We implemented fallback models and smoothing.
- **Hardware Constraints**: Operating within the <2GB RAM limit required careful feature selection and model optimization.
- **Rolling Predictions**: Maintaining consistency and avoiding error accumulation during rolling forecasts required custom loop structures and cache management.

11. Business Insights

Strategic Freezer Gambit Recommendations:

Strategy	Insight
Freeze High-Volatility Crops	e.g., Bottle Gourd in Starling City, save for peak weeks like Week 10–12.
Avoid Selling in High Rain Weeks	Correlation with the price drop was found in Mystic Falls. Freeze instead.
Capitalize on Predicted Peaks	If the forecast predicts a>15% increase, delay the sale using cold storage.
Expand Freezer Capacity	For regions with >30% week-to-week variance, extra storage prevents loss.

12. Conclusion

The AgroChill Forecasting Engine provides all of the necessary tools for accurate, low-resource, and scalable agricultural forecasting.It provides Agrovia stakeholders with actionable d ata into when to sell or store product based on expected market values.This system not only enab les strategic storage through cold chain infrastructure, but it also lays the groundwork for future modules such as optimisation and real-time analytics dashboards.

12. Appendices

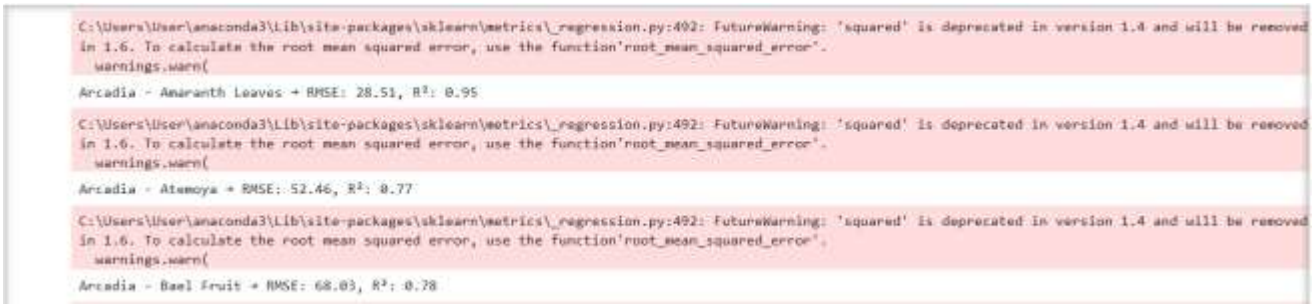
- Evaluating R^2 XGBRegressor

```
model.fit(X_train, y_train)

# ----------------------------------------
# Step 3: Evaluate on Evaluation Set
# ----------------------------------------
y_pred = model.predict(X_eval)
rmse = np.sqrt(mean_squared_error(y_eval, y_pred))
print(f"RMSE Value : {rmse:.4f}")

# ----------------------------------------
# Step 4: Save Trained Model
# ----------------------------------------
os.makedirs("model", exist_ok=True)
model.save_model("model/xgboost_model.json")
```

```
[1]  ✓ 12.1s

...  RMSE Value : 6.0660
```

```python
from sklearn.metrics import mean_squared_error, r2_score

# Predict on the test set
y_pred = model.predict(X_test)

# Evaluate using RMSE
rmse = mean_squared_error(y_test, y_pred, squared=False)

# Evaluate using R² score
r2 = r2_score(y_test, y_pred)

print(f"✅ RMSE: {rmse:.2f}")
print(f"✅ R² Score: {r2:.4f}")
```

```
✅ RMSE: 74.05
✅ R² Score: 0.7062
```



- Preprocessing Block

	Region	Commodity	Week	price	Temperature (K)
0	Arcadia	Amaranth Leaves	2040-01-02	283.89	304.15
1	Arcadia	Amaranth Leaves	2040-01-09	278.76	294.65
2	Arcadia	Amaranth Leaves	2040-01-16	98.04	293.95
3	Arcadia	Amaranth Leaves	2040-01-23	489.30	306.95
4	Arcadia	Amaranth Leaves	2040-01-30	61.76	300.45

	Rainfall (mm)	Humidity (%)	Crop Yield Impact Score	Year	Month
0	1333.2	81.0	1.68	2040	1
1	1038.6	60.0	2.10	2040	1
2	1656.0	81.8	0.76	2040	1
3	2667.6	95.4	1.03	2040	1
4	2149.8	52.4	2.08	2040	1

	Week_Num	Type
0	1	Vegetable
1	2	Vegetable
2	3	Vegetable
3	4	Vegetable
4	5	Vegetable